NazzOS — An Interactive Terminal-Based OS Learning & Simulation System

Project Report · Operating System Sessional **Course Code:** CIT-322

Student: Nazmus Sakib **ID:** 2102066 **Course Title:** Operating System Sessional

1. Introduction

NazzOS is a terminal-based, Python-powered platform built around one central idea: **learning OS concepts by doing them, not just reading about them.** Its primary feature is an embedded **Linux Learning System** — a structured, interactive curriculum covering 53 Linux commands across 9 topics, from basic navigation to advanced scripting. Alongside this, NazzOS includes a full-featured **OS Simulation Engine** that animates CPU scheduling, memory management, disk scheduling, deadlock detection, process synchronization, and a virtual file system — all live inside the terminal with no GUI required.

2. Primary Feature — Linux Learning System

The learning module (`modules/learn.py`) is what differentiates NazzOS from a plain simulator. A student can open NazzOS and learn Linux interactively, practice commands, and test knowledge — all without leaving the terminal.

Structured Curriculum — 9 Topics, 53 Commands

Topic Key	Title & Commands Covered	Cmds
`basic`	Basic Navigation — `pwd`, `ls`, `cd`, `mkdir`, `rmdir`, `clear`	6
`files`	File Operations — `touch`, `cat`, `cp`, `mv`, `rm`, `find`, `ln`	7
`text`	Text Processing — `grep`, `sort`, `wc`, `cut`, `sed`, `awk`, `head/tail`	7
`permissions`	Permissions & Ownership — `chmod`, `chown`, `umask`, `sudo`, `su`	5
`process`	Process Management — `ps`, `top`, `kill`, `jobs`, `bg/fg`, `nice`	6
`system`	System Information — `uname`, `df`, `du`, `free`, `uptime`, `who`, `env`	7
`network`	Networking — `ping`, `curl`, `wget`, `ssh`, `scp`, `netstat`, `ip`	7
`pipes`	Pipes & Redirection — `|`, `>`, `>>`, `<`, `tee`, `xargs`	6
`advanced`	Advanced Commands — `cron`, `tar`, `zip`, `tmux`, `alias`, bash scripts	7
	Total	**53**

Practice Challenges — 16 Tasks (Beginner → Advanced)

Each challenge presents a real-world task in plain English; the student types the correct Linux command and gets immediate feedback. A hint is available on request. Each correct answer scores 10 points; command variants (e.g. `ls -la` or `ls -al`) are accepted as equivalent.

Level	Sample Challenge & Answer
Beginner	*"Show your current directory path"* → `pwd`
Intermediate	*"Count lines in data.txt"* → `wc -l data.txt`
Advanced	*"Find all running Python processes"* → `ps aux | grep python`

Knowledge Quiz — 10 Multiple-Choice Questions

A randomized 10-question MCQ quiz with instant grading, correct-answer reveal, and a final percentage score with letter grade (A+ → F). Topics: directory navigation, redirection operators, process signals, permission notation, package managers, file comparison, and pipe usage.

Learning Shell Commands

Command	Action
`learn`	Show topic menu with all 9 topics
`learn basic` / `learn network` / …	Open topic — commands with examples & tips
`learn practice`	16-challenge practice mode with live scoring
`learn quiz`	10-question randomized MCQ quiz
`learn cheatsheet`	Quick-reference card for all 53 commands
`learn progress`	Show completed topics and current score

3. Additional Feature — OS Simulation Engine

Beyond learning, NazzOS provides animated, step-by-step simulation of six core OS topics, making it a complete teaching tool for any OS lab demonstration.

Module	Algorithms / Features	Visualization
CPU Scheduler `sched`	FCFS · SJF · SRTF · Round Robin · Priority · MLFQ	Animated Gantt chart; per-process WT & TAT; Avg WT, TAT, CPU util%, throughput; `sched compare` shows all 6 side-by-side
Memory Manager `mem`	First-Fit · Best-Fit · Worst-Fit; FIFO · LRU · Optimal · Clock; Buddy System	Color-coded frame grid (one color per process); hit/miss table; fault rate; buddy tree
Virtual File System `fs`	INode-based FS · FAT cluster chains · Unix permissions · `find` · `stat`	Rich tree view (📁 📄); inode metadata table; FAT linked-chain display; `chmod` output
Disk Scheduler `disk`	FCFS · SSTF · SCAN · C-SCAN · LOOK · C-LOOK	Animated head movement step-by-step; total & avg seek distance; compare table
Deadlock `deadlock`	Banker's Algorithm · Resource Allocation Graph · recovery suggestions	Allocation / Need / Available matrices; safe-sequence trace; RAG ASCII art
Synchronization `sync`	Dining Philosophers · Producer-Consumer · Reader-Writer · Semaphore	Philosopher state table (Thinking / Hungry / Eating); buffer slot animation
Live Dashboard `dash`	Real CPU, RAM, disk, process data via `psutil`	`rich.Live()` 4-panel: top processes, memory & CPU bars, FS tree, system info

4. Project Structure

```
mini-os/
├── main.py              # Entry point — boot animation → shell loop
├── shell.py             # prompt_toolkit REPL; routes all commands;
│                        # unknown commands forwarded to host OS (ls, cal …)
├── NazzOS.desktop       # Double-click launcher (Linux desktop shortcut)
├── ui/
│   ├── theme.py         # Color constants, process color cycling, rich Console
│   ├── boot.py          # ASCII 'NazzOS' logo + animated boot progress bar
│   └── widgets.py       # Shared helpers: Gantt chart, frame grid, bar chart
└── modules/
    ├── learn.py         # ★ PRIMARY — Linux Learning System
    │                    #   LESSONS (9 topics · 53 cmds)  CHALLENGES (16)
    │                    #   QUIZ (10 MCQ)  LearnModule class  handle_learn()
    ├── scheduler.py     # CPU scheduling (6 algos) + silent runners for compare
    ├── memory.py        # Allocation (3 algos) + page replacement (4 algos)
    │                    #   + buddy system + fragmentation stats
    ├── filesystem.py    # VirtualFS — INode dataclass, FAT clusters, chmod
    ├── disk.py          # DiskScheduler — 6 algorithms + head animation
    ├── deadlock.py      # DeadlockDetector — Banker's, RAG, recovery
```

```
    ├── sync.py          # Dining philosophers, producer-consumer, reader-writer
    └── dashboard.py     # rich Live() 4-panel real-time system dashboard
```

5. Sample Output



Figure 1: Linux Learning System — topic menu or practice mode

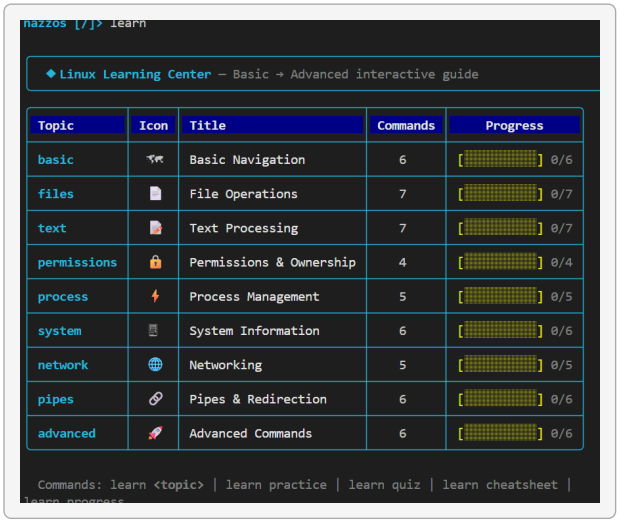


Figure 2: OS Simulation — scheduler Gantt or dashboard

6. Conclusion

NazzOS goes beyond a typical simulation project by embedding a complete, structured Linux Learning System at its core. A student can genuinely learn Linux here — following 9 progressive topics, completing 16 graded practice challenges, and taking a scored quiz — all without leaving the terminal. The OS simulation modules complement this by letting the student immediately observe the algorithms they study in animated, step-by-step form. Together, they make NazzOS a practical teaching platform that serves both learning and demonstration goals in a single unified tool.